
Team 14 Term Project: Developing a model that solves an easy-version of ARC-AGI

Jinsu Kim¹ Yongbeom Kim¹ Inho Kim¹ Junghwi Kim¹

1. Introduction

The Abstraction and Reasoning Corpus (ARC) is a challenging benchmark proposed by Chollet to measure a system’s ability to acquire “new” skills outside of its training data (ARC Prize, 2025). ARC tasks are visual puzzles presented as pairs of input-output grids composed of colored cells, with colors encoded by digits 0–9. Each task presents a grid-based visual puzzle where the system must analyze input-output examples to infer the underlying transformation rule, then apply this rule to solve test cases. Unlike conventional machine learning benchmarks, ARC emphasizes generalization to novel tasks: an intelligent agent must rapidly learn the underlying rule of each new puzzle from only a handful of demonstrations, reflecting a form of fluid intelligence. This makes ARC extremely challenging for current AI systems.

For this project, we work with an easy-version of the ARC-AGI challenge consisting of 380 tasks, where each task contains input-output pairs ranging from 100 to 1000 examples. Our approach centers on fine-tuning a relatively small Large Language Model (LLM), Alibaba’s Qwen2.5-3B-Instruct (Qwen Team, 2024). We use data augmentation to train the LLM on diverse data and improve inference performance (details in Section 3.2). In addition, we employ Low-Rank Adaptation (LoRA) for efficient fine-tuning (Hu et al., 2021) and reduce the model’s vocabulary to only the tokens needed for grid puzzles, both of which improve efficiency (details in Section 3.1, 3.3). Finally, at inference time we incorporate a depth-first search (DFS) to explore candidate outputs (details in Section 3.4). Using these techniques, our model achieved strong performance on our ARC-AGI dataset, with an estimated public leaderboard score approaching 85, and secured the 3rd place.

2. Exploratory Data Analysis

We first analyzed the characteristics of the ARC-AGI dataset prior to our model implementation. The ARC task is defined

over a grid-based world composed of colored cells. In our dataset, grid sizes vary, up to 10×10 , and there are 10 possible colors, represented by the digits 0 through 9. An example task is shown in Figure 1. All examples within a task share the same underlying transformation rule, mapping input grids to output grids accordingly.

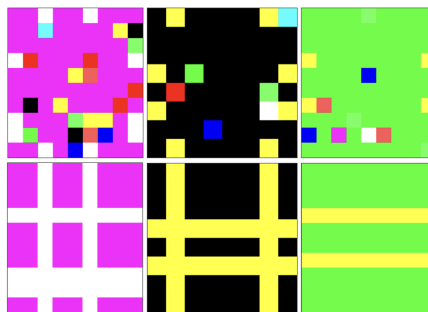


Figure 1. An example of an ARC task (6cdd2623) that demonstrates object expansion. In the input (upper grid), pixels that are different in color from the background and are located only at the edges and have pixels of the same color on the opposite edge are selected. The result of these pixels being expanded and connected by lines is shown in the output (lower grid).

Through manual inspection of the underlying rules applied to many tasks, we identified several recurring types of transformations or patterns:

1. Geometric transformations: Many puzzles involve spatial manipulations such as rotations, reflections, and translations of shapes.
2. Color mappings and swapping: Many puzzles are re-coloring certain parts of the input. Rules can map one color to another, or swap two colors, for pixels within a specific shape or an entire grid.
3. Cutting, merging, or morphing: Some tasks can require altering the shape of objects – for instance, splitting an object into pieces, merging multiple objects, shrinking or growing shapes, etc.
4. Object movement or extending: Some tasks’ rules are based on the concept of movement. For example, moving an object in one direction or extending a line until

^{*}Equal contribution ¹Department of Computer Science and Engineering, Seoul National University, Seoul, Korea.

it meets another line.

These observations guided several aspects of our method. First, the rules above suggest that we can augment the training data by applying such transformations without changing the underlying task rule. Indeed, applying transposition, rotation, and color permutation to the grid in Figure 1 does not change the rule. We leveraged this by generating augmented examples (details in Section 3.2). Second, the small vocabulary size (0-9 plus a few special symbols) means we can safely constrain the vocabulary of the tokenizer to only the necessary tokens (details in Section 3.1). Third, the maximum input size of the tasks is relatively limited compared to the original set, so the maximum token length of the model can be reduced for memory efficiency and stability. (details in Section 3.3). In summary, the ARC-AGI dataset’s structural properties present an opportunity to effectively augment data and optimize the model for this domain.

3. Method Pipeline

Our solution consists of a pipeline with four main components: choosing a suitable base model and customizing its tokenizer, augmenting the training data, fine-tuning the model on the ARC tasks, and a inference procedure that searches for the correct output via DFS algorithm. Figure 2 summarizes the overall pipeline. Our approach is based on the solution by the ARChitects and Guillermo Barbadillo, winners of the Arc prize 2024.

3.1. Model Selection and Tokenizer Reduction

We based our solver on the Qwen2.5-3B-Instruct model, a 3-billion-parameter transformer released by Alibaba in late 2024. This model was selected over alternatives (such as a LLaMA-3.2-3B model of similar size) for several reasons. First, we found examples of Qwen being used by winners of other ARC-AGI competitions (Barbadillo, 2024). Qwen models also emphasized fine-tuning in structured tasks related to pattern understanding and logical reasoning. (Qwen et al., 2025). These made Qwen2.5-3B a technically attractive choice for ARC-AGI.

After selecting the base model, we applied a custom tokenizer reduction to better adapt it to the ARC domain. The default Qwen2.5 tokenizer has a vocabulary of hundreds of thousands of tokens, mostly unused for our task (since the puzzles are essentially numeric and small textual markers). We pruned the vocabulary to retain only the characters relevant to grid representations and formatting, effectively reducing the model’s embedding matrix to a minimal set of ~ 80 tokens. This token pruning drastically reduces memory usage and concentrates the model’s representation capacity on the few symbols it actually needs to generate. We found this to be important when operating with limited VRAM

and using low-precision weights. As an additional benefit, the reduced vocabulary may slightly improve generation focus – the model is less likely to produce irrelevant text tokens and more likely to stay within the intended output format (grids of digits).

3.2. Data Augmentation Strategies

A core part of our approach is data augmentation, leveraging the fact that ARC tasks are invariant under certain transformations. Instead of training only on the original 380 tasks as given, we generate many modified versions of those tasks to provide additional training samples and to expose the model to a variety of input distributions. All augmentation operations are designed such that if the model learns a solution on the augmented task, it is directly relevant to solving the original task (i.e. we do not change the underlying rule, only the superficial appearance of the puzzle). We employed four main types of transformations:

1. Transposition: Reflecting the grids across the main diagonal, i.e. converting all (r, c) coordinates to (c, r) .
2. Rotation: Rotating all grids by 90, 180, or 270 degrees.
3. Color permutation: Applying a random permutation to the color labels (digits 0-9) in the entire task.
4. Example order shuffling: Randomly shuffling the order of the provided examples for a task in the prompt.

During training, we apply these augmentations probabilistically so that the size of the training dataset itself does not change. For each of the 380 base tasks, we created augmented versions by randomly deciding to transpose or not, picking a random rotation, generating a random color permutation, and shuffling the example order. Augmenting data in this way does not increase the amount of training data, but it can provide a more diverse distribution of data when training over multiple epochs.

During inference, the three examples given by default were augmented. Color permutations and example shuffles are applied in the same way as training, but transpositions and rotations are no longer probabilistic and all possible cases are generated. This augmentation makes it easier for our model to understand the rules of the task.

3.3. Training Configuration

To construct the prompts, we adopted a format similar to that used by the ARC prize 2024 winning solution (Daniel Franzen & Hartmann, 2024). In our format, each example’s input and output are presented text lines prefixed by $\mathbb{I}$ (meaning Input) and $\mathbb{O}$ (meaning Output), respectively. We also included a fixed pre-prompt string of letters $A \dots Z a \dots z$ excluding $\mathbb{I}$ and $\mathbb{O}$ at the very start of each prompt.

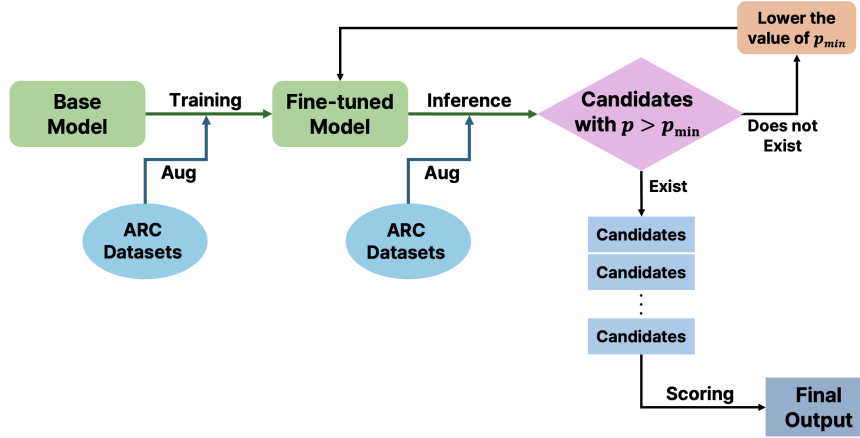


Figure 2. An overview of our pipeline. We fine-tune the model by training on the given ARC dataset from the base model. The trained model generates candidate(s) with probability p greater than the threshold $p_{\min}$ during the inference process. If there are such candidates, the most likely candidate is selected. If there are no strong candidates, the threshold is lowered and inference is performed again.

Table 1. Training configuration and hyperparameters for fine-tuning the Qwen2.5-3B model on ARC tasks. We utilized the Unsloth library for efficient LoRA training, which allowed low-precision fine-tuning with extended context lengths.

Hyperparameter	Value
Batch size	4
Gradient accumulation steps	2
LoRA rank	256
LoRA α	24
Learning rate (LoRA)	1e-4
Learning rate (Embedding)	1e-5
Learning rate schedule	Cosine decay
Learning rate warmup ratio	25%
Weight decay	0
Optimizer	AdamW, 8-bit variant
Training epochs	5

We fine-tuned the Qwen2.5-3B-Instruct model on the augmented ARC dataset using Low-rank adaptation (LoRA) via Unsloth library (Han et al., 2023). Table 1 summarizes our key hyperparameter settings. We set the LoRA rank to 128, which is a bit high. This is because the ARC-AGI training dataset not only demands the ability to discover new rules but also focuses on a highly specialized narrow domain. The maximum length of token sequence is 8192, based on training with 10×10 input and output grids.

The training process was tracked using Weights & Biases. The model was trained for 5 epochs, and the resulting fine-tuned LoRA weights were saved for inference.

3.4. Inference with Depth-First Search

To address the sensitivity of the task to individual token errors, we adopt a depth-first search (DFS)-based sampling algorithm. In our task, even a single incorrect token—corresponding to one pixel—can make the entire solution invalid. Therefore, instead of generating responses greedily or sequentially, our method explores multiple candidate completions by prioritizing tokens with higher certainty.

The steps of inference are as follows. At each step of the search, a single forward pass produces the logits for the next token. These logits are then passed through a softmax function to compute the probability p for each candidate token. If the probability for the i th token is p_i , the probability of generating up to N tokens is computed as $\prod_{i=1}^N p_i$. The certainty for the current response calculated in this way is compared with a preset threshold $p_{\min}$. If the certainty falls below the threshold, the path is excluded from the search, and only paths with a certainty exceeding $p_{\min}$ remain. In our code, the log value of the softmax is implemented as a score. The result of sampling is a list of candidate output grids, each with an associated score (likelihood).

We repeat this inference process multiple times with different initial conditions. First, we utilize the test-time augmentation discussed earlier and run DFS. In our implementation, we set $p_{\min} = 0.8$ for the first attempt. That is, we prune branches that have less than 80% probability. If first attempt yields no valid output, we lower the threshold gradually and retry. We used a schedule of $p_{\min} := 0.7, 0.5, 0.3, 0.1$ in subsequent attempts. This approach allows the model to search the high-confidence outputs at first; if no candidates are found, it expands the search to increasingly less likely

Table 2. Leaderboard scores by model improvements

MODEL	SCORE	TOTAL SCORE
BASE MODEL	7.00	7.00
+FINE TUNING (1 EPOCH)	+18.00	25.00
+DATA AUGMENTATION		
+TOKENIZER OPTIMIZATION		
+DFS	+31.00	56.00
+TEST-TIME DATA AUG.	+4.00	60.00
+FINE TUNING (5 EPOCH)	+13.00	83.00
+TEST-TIME TRAINING	+2.00	85.00

outputs.

This DFS algorithm allows us to search each pixel precisely while using memory efficiently. This is because the DFS algorithm does not store more answers than necessary to search.

4. Results

Our system achieves 85% accuracy on the competition leaderboard, securing third place among participants. This performance meets our expectations of 80-85% accuracy based on evaluation experiments. The final score is derived from our model with test-time training enabled, though we excluded TTT from official submission due to computational constraints.

Table 2 shows incremental improvements, demonstrating the effectiveness of each pipeline component. The largest gains come from data aug., tokenizer opt. and DFS (31 points), while TTT provide moderate but consistent improvements.

5. Discussions

5.1. Test-Time Training

Test-time training (TTT) refers to updating the model’s weights at inference time using the few examples of the task in the prompt before attempting to solve the test input. Recent research has shown TTT can significantly boost accuracy on ARC-AGI benchmark (Akyürek et al., 2025). So we integrated TTT into our model to trigger when the first inference fails, and that model achieved our best score by adapting the model to specific problem instances during inference. However, TTT introduced significant computational overhead approaching the 100-minute time limit, with several submissions experiencing timeouts. The modest performance gain (1-2 points) relative to computational cost led to the exclusion of TTT from our final submission. Future work should explore efficient implementations of TTT that maintain performance benefits within strict time constraints.

5.2. Concerns for Evaluation Validity

It seems like most of the leaderboard scores are scored on datasets that overlap with our training data, creating conditions where increased training data fitting directly improves measured performance rather than genuine generalization capability. This evaluation framework incentivizes overfitting to training examples, undermining the benchmark’s intended purpose of measuring abstract reasoning. Therefore, it may also be necessary to further reduce the proportion of training data in the test data for meaningful evaluation.

5.3. Refinements to DFS

There remains room for improvement in the current DFS-based scoring and sampling logic used in our approach. Specifically, the DFS scoring mechanism appears to introduce a positional bias, favoring continuations that occur later in the sequence. Conversely, the method may prematurely prune out promising candidate solutions that begin with low-probability tokens, thereby limiting the model’s ability to explore more diverse or less obvious paths. To mitigate these issues, incorporating a warmup phase with relaxed pruning thresholds could allow more exploratory depth early on. Alternatively, devising a scoring strategy for certainty of the model’s output may enhance solution coverage without additional computational cost.

6. Contribution

Jinsu Kim contributed to the reference research (The AR-Chitects, etc.), method design (data aug., DFS), coding (overall), training model, and wrote the outline of report.

Yongbeom Kim contributed to the reference research (TTT), method design (tokenizer opt., TTT), coding (data image maker, dataset loader, inference tools), and testing (tuning inference logic).

Junghwi Kim contributed to the model research, data analysis, coding (algorithmic solver), training model, presentation, and wrote section 4-5.

Inho Kim contributed to the data analysis, method design (algorithmic solving), coding (tokenizer opt.), and wrote section 1-3 (assisted writing section 4-5).

References

- Akyürek, E., Damani, M., Zweiger, A., Qiu, L., Guo, H., Pari, J., Kim, Y., and Andreas, J. The surprising effectiveness of test-time training for few-shot learning, 2025. URL <https://arxiv.org/abs/2411.07279>.
- ARC Prize, I. Arc-agi: About the benchmark, 2025. URL <https://arcprize.org/arc-agi>. Access: 2025-06-17.

Barbadillo, G. Solution summary, 2024. URL https://ironbar.github.io/arc24/05_Solution_Summary/. Access: 2025-06-17.

Daniel Franzen, J. D. and Hartmann, D. The llm architect: Solving arc-agi is a matter of perspective, 2024. URL https://github.com/da-fr/arc-prize-2024/blob/main/the_architects.pdf. Access: 2025-06-03.

Han, D., Han, M., and Unsloth team. Unsloth, 2023. URL <https://github.com/unslothai/unsloth>. GitHub repository.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models, 2021. URL <https://arxiv.org/abs/2106.09685>.

Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.